Redsand Protocol

Fixed-denomination private payments with claim NFTs,
off-chain Poseidon Merkle trees, and Groth16 withdraw proofs.

Network: Base Sepolia (chainId 84532)

Solidity: ^0.8.28

Core pattern: UUPS proxies + immutable root/verifier helpers

Source: hardhat-contracts/contracts

Document version 1.0

1. Overview

Redsand is a private payment pool. A user deposits a fixed ERC-20 amount, receives a claim NFT and keeps an off-chain Poseidon note secret, later generates a Groth16 membership proof, stages that proof on the NFT, and redeems for tokens minus a protocol fee.

Product flow

- deposit(commitment) - Vault pulls fixed denomination, mints PaymentNFT, emits Deposit
- Indexer inserts leaf into off-chain Poseidon tree and posts root to MerkleRootManager
- User proves with SDK/zk; stages proof on PaymentNFT via setRedemptionProof
- redeem(depositId) - Vault verifies proof, burns NFT, pays out (minus fee)

Contracts inventory

Vault: Core pool - deposit / redeem (UUPS)
PaymentNFT: Claim ticket + staged ZK proof (UUPS)
MerkleRootManager: On-chain root window of 64 (immutable)
FeeManager: Flat + bps fee configs (UUPS)
Treasury: Fee / fund custody (UUPS)
AdminController: Ops + upgrade hub (UUPS)
DepositVerifier: Mock verifier (always true) - replace in prod
WithdrawVerifier: Real Groth16 adapter implementing IVerifier
Groth16Verifier: snarkjs-generated pairing verifier
MockUSDC: Test ERC-20 (6 decimals)

Base Sepolia addresses

```
Vault               0x10E2A9407FdAB59AA249F5c41D85d992274C29ba
PaymentNFT          0x216E83e6Fc8bBE60B3dC4BF25B38122e29b88f14
MerkleRootManager   0xC514591ED8BccFde1A8c46Bb5663C499d39D8653
FeeManager          0x37Ee440fE247bB808FE0BB7E01FAA69f46960497
Treasury            0x23801F362eED288182A986bee5D41acC9f76fb9A
AdminController     0xCD9EF56FdF9132B66F9a01F9dCaBB273240F90C4
DepositVerifier     0x63FA4569Ae0DE03e5bdA224B85AB70Ba50aEDDE8
MockUSDC            0x814b263293965E51FBdF8c6A0D1de30689b5411e
```

2. Vault.sol

UUPS-upgradeable fixed-denomination private payment pool. Holds ERC-20 liquidity, mints claim NFTs on deposit, and pays out on redeem after ZK verification against a known Merkle root.

Upgradeability

- ERC1967 proxy + UUPS; upgrade authorized by owner (AdminController after deploy)
- Storage is append-only; 40-slot __gap reserved for future fields

Key state

- token / denomination - single configured ERC-20 and fixed amount
- verifier, rootManager, nftContract, feeManager, feeRecipient
- commitments, deposits, nullifierSpent, commitmentToDepositId
- depositCounter (leaf index), nextDepositId

User functions

- deposit(bytes32 commitment) - pulls denomination, mints NFT, records deposit, emits Deposit
- redeem(uint256 depositId) - requires NFT ownership + staged proof; verifies; burns; pays

Redeem checks (order)

- Deposit exists and not already redeemed
- msg.sender owns the claim NFT
- Proof staged (non-empty); nullifier and recipientDigest non-zero
- rootManager.isKnownRoot(root)
- nullifier not already spent
- verifier.verifyProof(... denomination, tokenHashOf(token))
- Mark redeemed + nullifier spent; burn NFT; transfer fee then payout

Owner / admin functions

- setToken(token, denomination) - once, before any deposits
- setNftContract / setFeeManager / setFeeRecipient / setVerifier / setRootManager
- pause / unpause
- emergencyWithdraw(token, to, amount) - only while paused

Events

```
Deposit(commitment, leafIndex, token, amount, depositId, tokenId)
Redeemed(depositId, nullifier, to, feeRecipient, fee, payout)
TokenConfigured / VerifierUpdated / RootManagerUpdated / EmergencyWithdraw
```

Custom errors

ZeroAddress, InvalidCommitment, CommitmentExists, TokenNotConfigured, NftNotConfigured, DepositMissing, AlreadyRedeemed, NotClaimOwner, NoProofStaged, RootNotFound, NullifierAlreadyUsed, InvalidProof, TokenAlreadySet, InvalidDenomination

3. PaymentNFT.sol

UUPS ERC-721 claim ticket. Vault mints on deposit; holder stages a redemption proof; Vault burns on successful redeem.

Roles

- DEFAULT_ADMIN_ROLE / ADMIN_ROLE / UPGRADER_ROLE - protocol admin
- VAULT_ROLE - granted via setVault; required for mintFor and burnFor

Functions

- setVault(address) - grants VAULT_ROLE to the Vault
- mintFor(to, depositId) - vault-only; returns tokenId; stores metadata mapping
- setRedemptionProof(tokenId, proof, root, nullifier, recipientDigest) - NFT owner only
- getRedemptionProof(tokenId) - returns staged proof fields
- burnFor(tokenId, caller) - vault-only; caller must be current owner
- setMetadataURI(tokenId, uri) - NFT owner

Staging validation

- proof.length > 0; nullifier != 0; recipientDigest != 0

4. MerkleRootManager.sol

Non-upgradeable Ownable2Step contract. Stores the last 64 posted Poseidon Merkle roots (Option B: tree is off-chain). The indexer/relayer owns this contract and calls postRoot. Vault only reads isKnownRoot.

Functions

- postRoot(bytes32 root) - owner-only; rejects zero/duplicate; ring buffer of 64
- isKnownRoot(root) - O(1) membership used by Vault.redeem
- latestRoot() / rootAt(i) / currentIndex() / postedCount()

Event (Goldsky)

```
RootPosted(bytes32 root, uint8 slot, uint256 postedCount)
```

5. FeeManager.sol

UUPS fee calculator used by Vault on redeem.

```
fee = flatFee + (amount * percentBps) / 10000
```

- setFeeConfig(token, flatFee, percentBps) - ADMIN_ROLE; percentBps <= 10000
- computeFee(token, amount) - token config if active, else address(0) default

Initialize sets a default address(0) config (commonly overridden for tests).

6. Treasury.sol

UUPS custody for protocol fees. Vault feeRecipient typically points here.

- GOVERNOR_ROLE: withdraw(token, to, amount)
- notifyReceived(token, amount) - emits Received
- balanceOf(token) - IERC20 balance helper

7. AdminController.sol

Top-level operations and upgrade hub. After deploy it owns the Vault (Ownable2Step) and holds UPGRADER_ROLE on FeeManager, PaymentNFT, and Treasury.

Roles

- DEFAULT_ADMIN_ROLE - wiring setters, acceptVaultOwnership
- OPERATOR_ROLE - unpause, fee config, vault config (verifier, token, etc.)
- EMERGENCY_ROLE - emergencyPauseAll, emergencyWithdrawFromVault
- UPGRADER_ROLE - upgradeVault / FeeManager / PaymentNFT / Treasury / self

Key operations

- emergencyPauseAll / unpauseAll - iterates pausableContracts
- setFeeConfig / setVerifier / setRootManager / setFeeRecipient / configureVaultToken
- upgradeVault(newImpl, data) and sibling upgrade* methods
- emergencyWithdrawFromVault(token, to, amount) - Vault must already be paused

8. Verifiers

IVerifier interface

Public input order is locked to the withdraw circuit (zk/PROTOCOL.md):

```
[0] root
[1] nullifier = Poseidon([secret])
[2] recipientDigest
[3] amount (= Vault.denomination)
[4] tokenHash = uint256(uint160(token))
```

DepositVerifier.sol (mock)

Always returns true. Currently deployed on Base Sepolia for early testing. Must be replaced before holding real value via AdminController.setVerifier.

WithdrawVerifier + Groth16Verifier

Production path. Groth16Verifier is the snarkjs-exported pairing check. WithdrawVerifier adapts ABI-encoded proof bytes and public inputs to IVerifier. Generated by zk/scripts/build.sh and export-verifier.mjs into contracts/verifiers/.

RejectingVerifier.sol

Test-only mock that always returns false (InvalidProof coverage).

9. MockUSDC.sol

Test ERC-20 with 6 decimals and owner mint. Used for Base Sepolia demos only.

10. Upgradeability matrix

Vault: UUPS - owner (AdminController)
PaymentNFT: UUPS - UPGRADER_ROLE
FeeManager: UUPS - UPGRADER_ROLE
Treasury: UUPS - UPGRADER_ROLE
AdminController: UUPS - UPGRADER_ROLE
MerkleRootManager: Not upgradeable - swap address on Vault
Verifiers: Not upgradeable - swap via setVerifier

11. Security notes

- Admin can pause and emergencyWithdraw - treat admin as trusted for hackathon
- Mock DepositVerifier accepts any proof - swap before real funds
- Merkle tree is off-chain; protect indexer RELAYER_PRIVATE_KEY (MerkleRootManager owner)
- Nullifiers prevent double-spend; commitments prevent duplicate deposits
- recipientDigest binds the note; ERC-20 payout goes to NFT owner (msg.sender)
- Fees are computed on-chain after proof verification (not in the circuit)

12. Goldsky indexing targets

For the deposit indexer, publish subgraphs only for:

- Vault - Deposit event (required)
- MerkleRootManager - RootPosted event (optional helper)

PaymentNFT, FeeManager, Treasury, AdminController, verifiers, and MockUSDC are not required for indexing.

13. Related off-chain packages

- zk/ - Circom withdraw circuit (Poseidon Merkle depth 20)
- sdk/ - note creation, Merkle helpers, Groth16 proving
- indexer/ - ingest deposits, postRoot, serve witnesses
- backend/ - wallet auth, usernames, encrypted notes (PostgreSQL)

End of document - Redsand Smart Contracts Specification v1.0